



第08章 多维数组和结构体

张仁斌@合肥工业大学软件学院

本章主要内容



8.1

定义二维数组



8.2

操作二维数组

8.3

二维数组作为函数参数

8.4

二维数组应用示例

8.5

多维数组

8.6

结构体

8.7

枚举和联合体

8.8

作业与实践



定义二维数组

◆ 定义二维数组的语法为：

元素数据类型 数组名[常量表达式][常量表达式]

行的数量

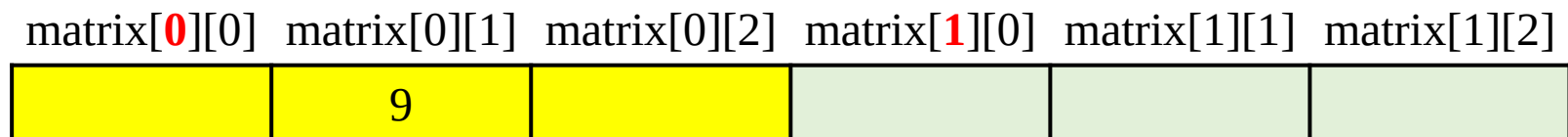
列的数量

行、列下标是从0开始的整数，
行、列下标必须分别用中括号，
不能采用“matrix[0, 1]”的形式

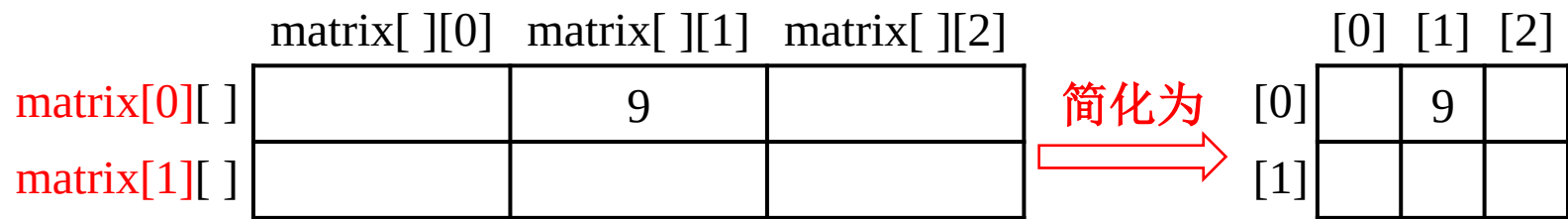
■ 例如：int matrix [2][3];

■ 利用行号和列号两个下标访问二维数组元素，例如：matrix[0][1] = 9;

■ 与一维数组一样，二维数组在**内存**中也是**连续**存储的：



从逻辑上理解为二维表格或矩阵：



可以把二维数组
看作一种特殊的一维数组，其元素又是一维数组

定义二维数组时初始化数组



- ◆ 与一维数组一样，也可以在一语句中定义并初始化二维数组，将初值依此写在花括号{ }内

```
int m[2][3] = {1, 2, 3, 4, 5, 6};
```

等价于（**分行/按行赋初值**）：

```
int m[2][3] = {{1, 2, 3}, {4, 5, 6}};
```

等价于

```
int m[2][3];
```

```
m[0][0] = 1; m[0][1] = 2; m[0][2] = 3;
```

```
m[1][0] = 4; m[1][1] = 5; m[1][2] = 6;
```

$$\begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix}$$

- ◆ 与一维数组一样，C++允许初始化二维数组时只给定部分元素，编译器自动给其余元素赋值0

```
int a[3][4] = {{1}, {5}, {9}};
```

$$\begin{bmatrix} 1 & 0 & 0 & 0 \\ 5 & 0 & 0 & 0 \\ 9 & 0 & 0 & 0 \end{bmatrix}$$
$$\begin{bmatrix} 1 & 0 & 0 & 0 \\ 5 & 6 & 0 & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix}$$

```
int b[3][4] = {{1}, {5, 6}};
```

```
int c[3][4] = {{1}, {}, {0, 0, 11}};
```

$$\begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 11 & 0 \end{bmatrix}$$
$$\begin{bmatrix} 1 & 0 & 6 & 7 \\ 8 & 9 & 0 & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix}$$

```
int d[3][4] = {1, 0, 6, 7, 8, 9};
```



◆ 若对全部元素都赋初值，或分行赋初值，则可以省略数组第一维长度，但数组第二维长度不允许省略

■ 全部元素均赋初值：

```
int m[3][4] = {1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12};  
等价于：  
int m[ ][4] = {1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12};
```

$$\begin{bmatrix} 1 & 2 & 3 & 4 \\ 5 & 6 & 7 & 8 \\ 9 & 10 & 11 & 12 \end{bmatrix}$$

■ 分行赋初值：

```
int a[ ][4] = {{0,0,3}, {}, {0,10}};
```

$$\begin{bmatrix} 0 & 0 & 3 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 10 & 0 & 0 \end{bmatrix}$$

本章主要内容



8.1 定义二维数组

8.2 操作二维数组



8.3 二维数组作为函数参数

8.4 二维数组应用示例

8.5 多维数组

8.6 结构体

8.7 枚举和联合体

8.8 作业与实践



对二维数组的常见操作

◆ 常用嵌套的for循环操作二维数组

◆ 基于定义如下的二维数组matrix介绍对二维数组的常见操作

```
const int ROW_SIZE = 10;  
const int COLUMN_SIZE = 10;  
int matrix[ROW_SIZE][COLUMN_SIZE];
```

◆ (1) 用输入值初始化二维数组

```
cout << "Enter " << ROW_SIZE << " rows and " << COLUMN_SIZE << " columns: " << endl;  
for(int i = 0; i < ROW_SIZE; i++)  
    for(int j = 0; j < COLUMN_SIZE; j++)  
        cin >> matrix[i][j];
```

◆ (2) 利用随机数初始化二维数组（假设限制随机数为0~99）

```
srand(time(0));  
for(int i = 0; i < ROW_SIZE; i++)  
    for(int j = 0; j < COLUMN_SIZE; j++)  
        matrix[i][j] = rand( ) % 100;
```



◆ (3) 输出二维数组（逐行输出）

```
for(int row = 0; row < ROW_SIZE; row++)  
{  
    for(int column = 0; column < COLUMN_SIZE; column++)  
        cout << matrix[row][column] << " "; // 元素的位数不同时，空格只能分隔元素，元素不能列对齐  
    cout << endl;  
}
```

◆ (4) 随机洗牌（随机交换二维数组中的任意两个元素）

```
srand(time(0));  
for(int row = 0; row < ROW_SIZE; row++)  
{  
    for(int column = 0; column < COLUMN_SIZE; column++)  
    {  
        int i = rand() % ROW_SIZE,    j = rand() % COLUMN_SIZE;  
        int temp = matrix[i][j];  
        matrix[i][j] = matrix[row][column];    matrix[row][column] = temp;  
    }  
}
```




◆ (5) 求全部元素的累加和

```
int sum = 0;
for(int row = 0; row < ROW_SIZE; row++)
    for(int column = 0; column < COLUMN_SIZE; column++)
        sum += matrix[row][column];
cout << "sum = " << sum << endl;
```

◆ (6) 求每一列的累加和

```
for(int column = 0; column < COLUMN_SIZE; column++)
{
    int sum = 0;
    for(int row = 0; row < ROW_SIZE; row++)
        sum += matrix[row][column];
    cout << "Sum for column " << column << " is " << sum << endl;
}
```



◆ (7) 求累加和最大的行号

如何求累加和最大的列号？

```
int maxRow = 0, indexOfMaxRow = 0;
for(int column = 0; column < COLUMN_SIZE; column++)
    maxRow += matrix[0][column];

for(int row = 1; row < ROW_SIZE; row++)
{
    int sumOfThisRow = 0;
    for(int column = 0; column < COLUMN_SIZE; column++)
        sumOfThisRow += matrix[row][column];
    if(sumOfThisRow > maxRow)
    {
        maxRow = sumOfThisRow;
        indexOfMaxRow = row;
    }
}
cout << "Row " << indexOfMaxRow << " has the max sum of " << sum << endl;
```

二维数组练习：代码的输出结果



```
#include <iostream>
using namespace std;

int main()
{
    int matrix[4][4] = {{1, 2, 3, 4},
                        {4, 5, 6, 7},
                        {8, 9, 10, 11},
                        {12, 13, 14, 15}};

    int sum = 0;
    for(int i = 0; i < 4; i++)
        sum += matrix[i][i];
    cout << sum << endl;

    for(int i = 0; i < 4; i++)
        cout << matrix[i][1] << " ";

    return 0;
}
```

D:\Edu-CPP\chap08\ex08-01.exe

```
31
2  5  9 13
```

本章主要内容



8.1 定义二维数组

8.2 操作二维数组

8.3 二维数组作为函数参数



8.4 二维数组应用示例

8.5 多维数组

8.6 结构体

8.7 枚举和联合体

8.8 作业与实践

二维数组作为函数参数



◆ 与一维数组一样，二维数组也可以作为函数参数，唯一不同之处是须在形参中给定列的长度

```
#include <iostream>
using namespace std;

const int COLUMN_SIZE = 4;

int sum(const int a[ ][COLUMN_SIZE], int rowSize)
{
    int total = 0;
    for (int row = 0; row < rowSize; row++)
        for (int column = 0; column < COLUMN_SIZE; column++)
            total += a[row][column];

    return total;
}
```

```
int main()
{
    const int ROW_SIZE = 3;
    int m[ROW_SIZE][COLUMN_SIZE];
    cout << "Enter " << ROW_SIZE << " rows and "
          << COLUMN_SIZE << " columns: " << endl;
    for (int i = 0; i < ROW_SIZE; i++)
        for (int j = 0; j < COLUMN_SIZE; j++)
            cin >> m[i][j];

    cout << "\nSum of all elements is "
          << sum(m, ROW_SIZE) << endl;

    return 0;
}
```

D:\Edu-CPP\chap08\ex08-02.exe

```
Enter 3 rows and 4 columns:
1 2 3 4
5 6 7 8
9 10 11 12
```

Sum of all elements is 78

构体

本章主要内容



8.1

定义二维数组

8.2

操作二维数组

8.3

二维数组作为函数参数

8.4

二维数组应用示例



8.5

多维数组

8.6

结构体

8.7

枚举和联合体

8.8

作业与实践

示例1：评定多项选择题测试成绩



- ◆ 假设8位学生回答10道多项选择题的答案保存在一个二维数组中，数组每行记录一位学生的全部答案

	0	1	2	3	4	5	6	7	8	9
学生0	A	B	A	C	C	D	E	E	A	D
学生1	D	B	A	B	C	A	E	E	A	D
学生2	E	E	D	A	C	B	E	E	A	D
学生3	C	B	A	E	D	C	E	E	A	D
学生4	A	B	D	C	C	D	E	E	A	D
学生5	B	B	E	C	C	D	E	E	A	D
学生6	B	B	A	C	C	D	E	E	A	D
学生7	E	B	E	C	C	D	E	E	A	D

- ◆ 正确答案存储在一个一维数组中

	0	1	2	3	4	5	6	7	8	9
答案	D	B	D	C	C	D	A	E	A	D



◆ 编写程序统计每位学生答对题数，并输出结果

```
#include <iostream>
using namespace std;

int main() {
    const int STUDENTS = 8;
    const int QUESTIONS = 10;

    char answers[STUDENTS][QUESTIONS] =
    {
        {'A', 'B', 'A', 'C', 'C', 'D', 'E', 'E', 'A', 'D'},
        {'D', 'B', 'A', 'B', 'C', 'A', 'E', 'E', 'A', 'D'},
        {'E', 'D', 'D', 'A', 'C', 'B', 'E', 'E', 'A', 'D'},
        {'C', 'B', 'A', 'E', 'D', 'C', 'E', 'E', 'A', 'D'},
        {'A', 'B', 'D', 'C', 'C', 'D', 'E', 'E', 'A', 'D'},
        {'B', 'B', 'E', 'C', 'C', 'D', 'E', 'E', 'A', 'D'},
        {'B', 'B', 'A', 'C', 'C', 'D', 'E', 'E', 'A', 'D'},
        {'E', 'B', 'E', 'C', 'C', 'D', 'E', 'E', 'A', 'D'}
    };
};
```

```
char keys[] = {'D', 'B', 'D', 'C', 'C', 'D', 'A', 'E', 'A', 'D'};

for (int i = 0; i < STUDENTS; i++)
{
    int correctCount = 0;
    for (int j = 0; j < QUESTIONS; j++)
    {
        if (answers[i][j] == keys[j])
            correctCount++;
    }
    cout << "学生" << i << "答对题数: " << correctCount << endl;
}

return 0;
}
```

如何按答对题
数升序输出？

D:\Edu-CPP\chap08\ex08-03.exe

```
学生0答对题数: 7
学生1答对题数: 6
学生2答对题数: 5
学生3答对题数: 4
学生4答对题数: 8
学生5答对题数: 7
学生6答对题数: 7
学生7答对题数: 7
```




示例2：求最邻近点

- ◆ 给定用x、y坐标确定的点集，寻找其中距离最近的两个点
 - 求点对之间的距离，然后找出最小距离及其对应的点对

```
#include <iostream>
#include <cmath>
using namespace std;

// Compute the distance between two points (x1, y1) and (x2, y2)
double getDistance(double x1, double y1, double x2, double y2)
{
    return sqrt((x2 - x1) * (x2 - x1) + (y2 - y1) * (y2 - y1));
}
```

	x	y
0	-1	3
1	-1	-1
2	1	1
3	2	0.5
4	2	-1
5	3	3
6	4	2
7	4	-0.5

```

int main()
{
    const int NUMBER_OF_POINTS = 8;

    // Each row in points represents a point
    double points[NUMBER_OF_POINTS][2];

    cout << "Enter " << NUMBER_OF_POINTS << " points: ";
    for (int i = 0; i < NUMBER_OF_POINTS; i++)
        cin >> points[i][0] >> points[i][1];

    // p1 and p2 are the indices in the points array
    int p1 = 0, p2 = 1; // Initial two points
    double shortestDistance = getDistance(
        points[p1][0], points[p1][1],
        points[p2][0], points[p2][1]);

```

 D:\Edu-CPP\chap08\ex08-04.exe

```

Enter 8 points: -1 3 -1 -1 1 1 2 0.5
2 -1 3 3 4 2 4 -0.5
最邻近两点是(1, 1)和(2, 0.5)

```

```

// Compute distance for every two points
for (int i = 0; i < NUMBER_OF_POINTS; i++)
{
    for (int j = i + 1; j < NUMBER_OF_POINTS; j++)
    {
        double distance = getDistance(
            points[i][0], points[i][1],
            points[j][0], points[j][1]);

        if (shortestDistance > distance)
        { // Update shortestDistance
            p1 = i; // Update p1
            p2 = j; // Update p2
            shortestDistance = distance;
        }
    }
}

cout << "最邻近两点是(" << points[p1][0] << ", "
    << points[p1][1] << ")和("
    << points[p2][0] << ", " << points[p2][1] << ")";
return 0;
}

```

如何获取距离都为某
最小值的全部邻近点



示例3：矩阵转置

◆ 已知行列的矩阵转置： $a = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix}$ 转换成 $b = \begin{bmatrix} 1 & 4 \\ 2 & 5 \\ 3 & 6 \end{bmatrix}$

```
#include <iostream>
#include <iomanip>
using namespace std;

int main() {
    int a[2][3]={ {1, 2, 3}, {4, 5, 6} };
    int b[3][2], i, j;
    cout << "array a: \n";
    for(i = 0; i <= 1; i++)
    {
        for(j = 0; j <= 2; j++)
        {
            cout << setw(5) << a[i][j];
            b[j][i] = a[i][j];
        }
        cout<<"\n";
    }
}
```

```
cout << "\narray b: \n";
for(i = 0; i <= 2; i++)
{
    for(j = 0; j <= 1; j++)
        cout<<setw(5)<<b[i][j];
    cout<<"\n";
}
return 0;
}
```

```
D:\Edu-CPP\chap08\ex08-05.exe

array a:
    1    2    3
    4    5    6

array b:
    1    4
    2    5
    3    6
```

本章主要内容



8.1 定义二维数组

8.2 操作二维数组

8.3 二维数组作为函数参数

8.4 二维数组应用示例

8.5 多维数组



8.6 结构体

8.7 枚举和联合体

8.8 作业与实践



多维数组的定义

- ◆ 在C/C++中可以创建任意维数的数组
- ◆ 与一维、二维数组的定义相似，定义多维数组的语法为
元素数据类型 数组名[常量表达式][常量表达式][常量表达式]...
- ◆ 例如：假设6位学生学习5门课程，每门课程的成绩包括平时成绩和期末考试成绩两部分，则可以定义一个三维数组存储成绩清单：

哪位学生

哪门课

是平时成绩还是期末考试成绩

```
double scores[6][5][2] = {  
    {{75, 80.5}, {90, 85.5}, {90, 91.5}, {85, 92.5}, {95, 90.5}},  
    {{90, 92.5}, {70, 81.5}, {85, 97.5}, {80, 87.5}, {90, 77.5}},  
    {{65, 84.5}, {60, 65.5}, {70, 72.5}, {75, 81.5}, {65, 69.5}},  
    {{70, 79.5}, {75, 93.5}, {90, 98.5}, {85, 88.5}, {80, 92.5}},  
    {{75, 86.5}, {70, 87.5}, {70, 74.5}, {80, 89.5}, {75, 80.5}},  
    {{80, 93.5}, {90, 94.5}, {85, 95.5}, {85, 83.5}, {70, 87.5}}};
```

示例：计算每天的平均温度和湿度



◆气象站每天每隔1小时记录1次温度和湿度。文档weather.txt中存储了此前10天记录的数据，文件的每一行由四个数字组成，分别表示日期（1~10）、时间（1~24）、温度和湿度。计算每天的平均温度和湿度

◆分析

■定义三维数组data[10][24][2]，使用输入重定向从文件weather.txt读取记录，存入数组

■data第一个下标范围是0~9，对应10天；第二个下标范围是0~23，对应24小时；第三个下标范围是0~1，分别对应温度和湿度，而文件中的日期是1~10，小时是1~24

➢ data[0][0][0]存储第1天第1小时的温度，data[9][23][1]存储第10天第24小的湿度

哪一天

哪个小时

是温度还是湿度

```
double data[10][24][2] = {.....};
```

```

#include <iostream>
using namespace std;

int main()
{
    const int DAYS = 10;
    const int HOURS = 24;
    double data[DAYS][HOURS][2] = {0};

    // Read input using input redirection from a file
    int day, hour;
    double temperature, humidity;
    for (int k = 0; k < DAYS * HOURS; k++)
    {
        cin >> day >> hour >> temperature >> humidity;
        data[day - 1][hour - 1][0] = temperature;
        data[day - 1][hour - 1][1] = humidity;
    }

```

```

// Find the average daily temperature and humidity
for (int i = 0; i < DAYS; i++)
{
    double dailyTemperatureTotal = 0;
    double dailyHumidityTotal = 0;
    for (int j = 0; j < HOURS; j++)
    {
        dailyTemperatureTotal += data[i][j][0];
        dailyHumidityTotal += data[i][j][1];
    }

    // Display result
    cout << "第" << i << "天平均温度是"
        << dailyTemperatureTotal / HOURS
        << ", 平均湿度是"
        << dailyHumidityTotal / HOURS << endl;
}

return 0;
}

```

C:\WINDOWS\system32\cmd.exe

```

D:\Edu-CPP\chap08>ex08-06.exe < weather.txt
第0天平均温度是77.7708, 平均湿度是0.929583
第1天平均温度是77.3125, 平均湿度是0.929583
第2天平均温度是77.6458, 平均湿度是0.929583
第3天平均温度是77.6458, 平均湿度是0.929583
第4天平均温度是77.6458, 平均湿度是0.929583
第5天平均温度是77.6458, 平均湿度是0.929583
第6天平均温度是77.6458, 平均湿度是0.929583
第7天平均温度是77.6875, 平均湿度是0.929583

```

本章主要内容



8.1 定义二维数组

8.2 操作二维数组

8.3 二维数组作为函数参数

8.4 二维数组应用示例

8.5 多维数组

8.6 结构体



8.7 枚举和联合体

8.8 作业与实践

结构体



- ◆ 数组是**同类型数据**的集合
- ◆ 在实际生活中经常会遇到一组类型不同的相关数据，例如，一个学生的学号、姓名、性别、年龄、成绩、家庭地址等项

id	name	sex	age	score	address
20240001	Wang Wu	M	19	90.5	Hefei

- 为了方便处理信息，将这些信息定义为一个集合。为此，C++给出了一种**构造数据类型**——“结构体”（**不同类型数据的集合**）

结构体类型与结构体变量



◆ 结构体类型声明

struct 结构体名

```
{  
    类型标识符 成员名1;  
    类型标识符 成员名2;  
    .....  
};
```

日期 struct date

```
{  
    int year;    // 年  
    int month;   // 月  
    int day;     // 日  
};
```

学生 struct student

```
{  
    int id;           // 学号  
    char name[20];    // 姓名  
    char sex;         // 性别  
    struct date birthday; // 生日  
    float score[4];   // 分数  
};
```

- 结构体类型定义结构的**组织形式**，**不分配内存**
- 结构体由多个组织在一起的数据项组成，这些数据项不需要属于同一类型
- 结构体可以容纳需要的任意多数据项
- 结构体中的变量称为结构体元素或结构体成员

◆ 结构体变量定义

struct date day;

struct student stu1, stu2;

```
struct date {  
    int year;  
    int month;  
    int day;  
} day2;
```



结构体变量初始化

- ◆ 系统按每个成员在结构体中的顺序一一对应赋初值。若只对部分成员进行初始化，则只能给前面的若干成员赋值，而不允许跳过前面的成员给后面的成员赋值。未赋值部分均为0值

```
struct date
```

```
{  
    int year;  
    int month;  
    int day;
```

```
};
```

```
struct date day = {2024, 11, 25};
```

```
struct date
```

```
{  
    int year;  
    int month;  
    int day;
```

```
} day = {2024, 11, 25};
```

```
struct student
```

```
{  
    int num;  
    char name[20];  
    char sex;  
    struct date birthday;  
    float score[4];
```

```
};
```

```
struct student stu = {20240001, "Wang Wu", 'M', 2005, 9, 10, 90.5};
```

- ◆ 只可以在初始化时整体赋值

```
struct student stu;
```

```
stu = {20240001, "Wang Wu", 'M', 2005, 9, 10, 90.5}; ❌
```



结构体变量访问—成员操作符

◆访问方式：结构体变量名.成员名 成员运算符，结合性：从左向右

◆访问规则：不能整体引用，只能引用变量成员

```
struct date
{
    int year;
    int month;
    int day;
};
```

```
struct student
{
    int id;
    char name[20];
    char sex;
    struct date birthday;
    float score[4];
};
```

```
struct student stu1;
stu1.id = 20240001;
strcpy(stu1.name, "Wang Wu");
stu1.score[0] = 90.5;
stu1.birthday.month = 9;
```

name是C字符串，须使用字符串操作函数

score是数组，非定义时初始化，须下标

birthday是结构体，只能对其成员初始化

```
struct student stu = {20240001, "Wang Wu", 'M', 2005, 9, 10, 90.5};
```

利用指向结构的指针来访问成员：

```
struct student stu1, *ptr;
ptr = &stu1;
ptr->id = 10;
ptr->score[0] = 90.5;
(*ptr).id += 1;
```

以下表示结构成员的形式是等效的：

- ①结构变量.成员名
- ②(*结构指针变量).成员名
- ③结构指针变量->成员名



◆ 注意事项

- 不能将结构体变量作为一个整体进行输入和输出

```
cin >> stu1;
```

- 成员本身是结构体类型时，只能对最低级的成员进行赋值等运算

```
stu1.birthday.month = 9;
```

- 结构体成员与普通变量一样，可以进行各种运算

```
stu1.score[0] = 80.5;  
stu2.score[0] = 87.5;  
int i = stu1.score[0] + stu2.score[0];
```

```
struct date {  
    int year;  
    int month;  
    int day;  
};  
struct student {  
    int id;  
    char name[20];  
    char sex;  
    struct date birthday;  
    float score[4];  
};  
  
struct student stu1, stu2;
```



结构体数组

- ◆ 首先声明结构体类型，然后定义该类型的数组变量

```
struct student stu[5];
```

- ◆ 访问数组

```
stu[2].id = 20240001;
```

数组stu第三个元素的成员变量id

```
struct date {  
    int year;  
    int month;  
    int day;  
};  
struct student {  
    int id;  
    char name[20];  
    char sex;  
    struct date birthday;  
    float score[4];  
};  
  
struct student stu1, stu2;
```

示例：设计一个函数统计候选人得票



```
#include<iostream>
#include<cstring>
#include<iomanip>
using namespace std;
```

```
struct candidate {
    char name[20];
    int count;
};
```

C中须是“**struct** candidate a[]”，
C++中可以省略struct

```
void votes(candidate a[ ], int n) {
    char str[20];
    cin >> str;
    while (strcmp(str, "end"))
    {
        for(int i = 0; i < n; i++)
            if(!strcmp(str, a[i].name)) a[i].count++;
        cin >> str;
    }
}
```

输入为“end”时strcmp()
返回0，否则非零

```
int main()
{
    const int SIZE = 3;
    candidate ca[SIZE]={"zhang", 0, "wang", 0, "liu", 0};
    votes(ca, SIZE);
    cout << "投票结果为: \n";
    for(int i = 0; i < SIZE; i++)
        cout << setw(5) << ca[i].name <<": " << ca[i].count << endl;

    return 0;
}
```

D:\Edu-CPP\chap08\ex08-07.exe

```
zhang
zhang
wang
end
投票结果为:
zhang: 2
wang: 1
liu: 0
```

本章主要内容



8.1 定义二维数组

8.2 操作二维数组

8.3 二维数组作为函数参数

8.4 二维数组应用示例

8.5 多维数组

8.6 结构体

8.7 枚举和联合体

8.8 作业与实践



枚举



◆ 日常生活中的枚举

- 性别：男，女
- 季节：春，夏，秋，冬

◆ 如果变量取值是有限种可能，则可声明一个**枚举类型**，将变量的取值一一列举出来：

enum 枚举类型名 {枚举值1, 枚举值2, ...};

枚举值是**常量**，系统定义了表示其序号的数值，从0开始，枚举值按顺序定义为0, 1, 2, ...

```
enum sex{MALE, FEMALE};  
enum color{RED, YELLOW, BLUE, WHITE, BLACK};  
enum weekdays {SUNDAY, MONDAY, TUESDAY, WEDNESDAY, THURSDAY, FRIDAY, SATURDAY};
```

MALE = 3; FEMALE = 4; // 给常量赋值 ❌

■ 枚举元素的值可以在定义时人为改变（非默认的0、1、2、.....）

```
enum weekdays {SUNDAY = 7, MONDAY = 1, TUESDAY, WEDNESDAY, THURSDAY, FRIDAY, SATURDAY};
```

TUESDAY未指定值，按前邻元素值递增1为2，后续依次递增1



枚举类型与枚举变量

◆ 先定义枚举类型，再定义该类型的变量

```
enum weekdays {SUNDAY, MONDAY, TUESDAY, WEDNESDAY, THURSDAY, FRIDAY, SATURDAY};  
enum weekdays workday; // 定义枚举类型变量
```

◆ 只能把枚举元素赋给枚举变量，不能把与枚举元素相当的数值直接赋给枚举变量，即一个整数不能直接赋值给一个枚举变量

```
workday = 2; ✗  
workday = TUESDAY;
```

■ 若一定要把数值赋给枚举变量，则必须用强制类型转换

```
workday = (enum weekdays)6; // 相当于workday = SATURDAY;
```

枚举类型应用示例



- ◆ 口袋中有红、黄、蓝、白、黑五种颜色的球若干个，每次从口袋中取三个不同颜色的球，统计并输出所有的取法
- ◆ 分析
 - 五种颜色可用枚举类型表示
 - 设取出球i、j、k，根据题意，i、j、k各自有五种取法，且 $i \neq j \neq k$
 - 可以用穷举法，逐个检验每一种可能的组合，从中找出符合要求的组合并输出

```
#include<iostream>
#include<iomanip>
using namespace std;
```

```
int main()
{
    enum color_set {RED, YELLOW, BLUE, WHITE, BLACK};
    char color_name[ ][7] = {"red", "yellow", "blue", "white", "black"};
    color_set color;
    int i, j, k, counter = 0;
```

D:\Edu-CPP\chap08\ex08-08.exe

```
1:      red      yellow      blue
2:      red      yellow      white
3:      red      yellow      black
4:      red      blue       yellow
5:      red      blue       white
6:      red      blue       black
7:      red      white      yellow
8:      red      white      blue
9:      red      white      black
10:     red      black      yellow
11:     red      black      blue
12:     red      black      white
13:     yellow    red       blue
14:     yellow    red       white
```

```
for(i = RED; i <= BLACK; i++)
{
    for(j = RED; j <= BLACK; j++)
    {
        if(i != j)
        {
            for(k = RED; k <= BLACK; k++)
                if(k != i && k != j)
                {
                    cout << setw(2) << ++counter << ":";
                    cout << setw(10) << color_name[i];
                    cout << setw(10) << color_name[j];
                    cout << setw(10) << color_name[k];
                    cout << endl;
                }
            }
        }
    }
    return 0;
}
```



联合体

◆ 联合体类型声明

union 联合体名称

```
{  
    类型标识符  成员名1;  
    类型标识符  成员名2;  
    .....  
};
```

```
union mark {  
    char grade; // 等级制  
    bool pass;  // 二级制  
    int percent; // 百分制  
};
```

■ 特点:

- 成员共用相同的内存单元（编译器只为最大的成员分配足够大的内存单元，全部成员共用该内存单元），联合体也称作**共用体**
- 任何两个成员不会同时有效

◆ 联合体变量定义

union mark a, b;

```
union mark {  
    char grade;  
    bool pass;  
    int percent;  
} a, b;
```



◆ 联合体变量初始化

```
union mark {  
    char grade; // 等级制  
    bool pass;  // 二级制  
    int percent; // 百分制  
};  
  
mark a;  
a.grade = 'A';  
a.percent = 100; // 覆盖grade  
mark b = a;      // 用一个联合初始化为另一个同类型的联合  
mark c = {'A'};  // 初始化联合的第一个成员（若列举多个值，编译报错）
```

◆ 不管怎么初始化和赋值，最终都有且仅有一个成员是有效的

结构体、枚举、联合体综合应用示例



◆ 编写程序录入试卷上的课程信息（课程名称、学生姓名、记分模式和成绩），并输出

■ 记分模式有等级制、二级制和百分制三种

```
#include <string>
#include <iostream>
#include <iomanip>
using namespace std;

// 记录成绩三种模式
enum scoreMode{GRADE, PASS, PERCENTAGE};

// 成绩
union score
{
    char grade; // 等级制成绩
    bool pass; // 是否通过课程
    int percent; // 百分制成绩
};
```

```
// 课程信息
struct courseInfo
{
    string course_name;
    string student_name;
    enum scoreMode mode;
    union score score_value;
};
```

```
void show(courseInfo info)
{
    cout << setw(15) << "课名称: " << info.course_name << endl;
    cout << setw(15) << "学生姓名: " << info.student_name << endl;
    cout << setw(15);
    switch (info.mode)
    {
        case GRADE:
            cout << "成绩(等级制): " << info.score_value.grade;
            break;
        case PASS:
            cout << "成绩(二级制): " << (info.score_value.pass ? "PASS" : "FAIL");
            break;
        case PERCENTAGE:
            cout << "成绩(百分制): " << info.score_value.percent;
            break;
    }
    cout << endl;
}
```



```
int main()
{
    courseInfo info;
    int i;
    cout << "请分别输入课程名称, 学生姓名, 成绩模式: \n";
    getline(cin, info.course_name);
    getline(cin, info.student_name);
    cin >> i;
    info.mode = (enum scoreMode)i;
    switch (info.mode)
    {
        case GRADE:
            cout << "输入等级制成绩: \n"; cin >> info.score_value.grade;
            break;
        case PASS:
            cout << "输入二级制成绩: \n"; cin >> info.score_value.pass;
            break;
        case PERCENTAGE:
            cout << "输入百分制成绩: \n"; cin >> info.score_value.percent;
    }
    show(info);
    return 0;
}
```

D:\Edu-CPP\chap08\ex08-09.exe

```
请分别输入课程名称, 学生姓名, 成绩模式:
C++
zhang san
2
输入百分制成绩:
96

          课名称: C++
          学生姓名: zhang san
          成绩(百分制): 96
```

本章主要内容



8.1 定义二维数组

8.2 操作二维数组

8.3 二维数组作为函数参数

8.4 二维数组应用示例

8.5 多维数组

8.6 结构体

8.7 枚举和联合体

8.8 作业与实践



第8章作业



(1)编写函数，返回一个矩阵指定列的全部元素之和，函数头如下：

```
const int SIZE = 4;  
double sumColumn(const double m[ ][SIZE], int rowSize, int columnIndex);
```

编写测试程序读入 3×4 的矩阵，输出其每列的元素之和

(2)编写函数求 $n \times n$ 双精度方阵主对角线上的元素之和，函数头如下：

```
const int SIZE = 4;  
double sumMajorDiagonal(const double m[ ][SIZE]);
```

编写测试程序读入 4×4 的方阵，输出其主对角线上的元素之和

(3)编写程序随机用0和1填充一个 4×4 的方阵，输出该方阵，并找出全为0或1的行、列和对角线



(4)假设所有员工每周工作小时数保存在一个二维数组中，每行保存一个员工7天的工作时数。例如下面数组保存了8位员工的周工作时数。编写程序按周工作总时数递减顺序输出全部员工及其周工作时数

	周日	周一	周二	周三	周四	周五	周六
员工0	2	4	3	4	5	8	8
员工1	7	3	4	3	3	4	4
员工2	3	3	4	3	3	2	2
员工3	9	3	4	7	3	4	1
员工4	3	5	4	3	6	3	8
员工5	3	4	4	6	3	4	4
员工6	3	7	4	8	3	8	4
员工7	6	3	5	9	2	7	9



(5)编写函数实现两个矩阵的**a**和**b**相乘，结果存储在矩阵**c**中。

$$\begin{bmatrix} a_{11} & a_{12} & a_{13} \\ a_{21} & a_{22} & a_{23} \\ a_{31} & a_{32} & a_{33} \end{bmatrix} \times \begin{bmatrix} b_{11} & b_{12} & b_{13} \\ b_{21} & b_{22} & b_{23} \\ b_{31} & b_{32} & b_{33} \end{bmatrix} = \begin{bmatrix} c_{11} & c_{12} & c_{13} \\ c_{21} & c_{22} & c_{23} \\ c_{31} & c_{32} & c_{33} \end{bmatrix}$$

函数头为：

```
const int N = 3;
```

```
Void multiplyMatrix(const double a[ ][N], const double b[ ][N], double c[ ][N]);
```

每个元素 $c_{ij} = a_{i1} \times b_{1j} + a_{i2} \times b_{2j} + a_{i3} \times b_{3j}$ 。编写测试程序提示用户输入两个 3×3 矩阵，输出其乘积

(6)编写程序随机用0和1填充一个 4×4 矩阵，输出该矩阵，并找出有最多1的第一个行和列，输出该行号和列号



(7)编写函数`void sort(int m[ ][2], int numberOfRows)`排序一个二维数组，该函数对每一行按照第一个元素排序，若第一个元素相同，则按第二个元素排序。例如，数组`{{4, 2}, {1, 7}, {4, 5}, {1, 2}, {1, 1}, {4, 1}}`将被排序为`{{1, 1}, {1, 2}, {1, 7}, {4, 1}, {4, 2}, {4, 5}}`。编写测试程序提示输入10个坐标点，调用该函数输出排序后的点

(8)编写函数`bool isConsecutiveFour(int values[ ][7])`用于检查一个二维数组是否在水平、垂直或对角方向上含有连续4个相同的数字。编写测试程序提示输入二维数组的行数和列数以及数组元素值。若该数组含有4个连续相同的数字，则输出为真，否则输出为假。为真的例子如：

0	1	0	3	1	6	1
0	1	6	8	6	0	1
5	6	2	1	8	2	9
6	5	6	1	1	9	1
1	3	6	1	4	0	7
3	3	3	3	4	0	7

0	1	0	3	1	6	1
0	1	6	8	6	0	1
5	5	2	1	8	2	9
6	5	6	1	1	9	1
1	5	6	1	4	0	7
3	5	3	3	4	0	7

0	1	0	3	1	6	1
0	1	6	8	6	0	1
5	6	2	1	6	2	9
6	5	6	6	1	9	1
1	3	6	1	4	0	7
3	6	3	3	4	0	7

0	1	0	3	1	6	1
0	1	6	8	6	0	1
9	6	2	1	8	2	9
6	9	6	1	1	9	1
1	3	9	1	4	0	7
3	3	3	9	4	0	7



(9)若 $n \times n$ 矩阵的每个元素都大于0且每列全部元素累加和为1，则该矩阵为马尔科夫矩阵。编写函数检查一个矩阵是否为马尔科矩阵：

```
const int SIZE = 3;
```

```
bool isMarkovMatrix(const double m[ ][SIZE]);
```

编写测试程序提示输入一个 3×3 的双精度矩阵，检查其是否是马尔科夫矩阵。输出示例如下：

```
Enter a 3 by 3 matrix row by row:
```

```
0.15 0.875 0.375
```

```
0.55 0.005 0.225
```

```
0.30 0.12 0.4
```

```
It is a Markov matrix
```

```
Enter a 3 by 3 matrix row by row:
```

```
0.95 -0.875 0.375
```

```
0.65 0.005 0.225
```

```
0.30 0.22 -0.4
```

```
It is not a Markov matrix
```



(10)实现下面对一个二维数组的列进行排序的函数，返回一个排序后的新数组，不改变输入数组：

```
const int SIZE = 3;
```

```
void sortColumns(const double m[ ][SIZE], double result[ ][SIZE]);
```

编写测试程序提示输入一个 3×3 的双精度矩阵，输出排序后的矩阵。运行示例如下：

Enter a 3 by 3 matrix row by row:

0.15 0.875 0.375

0.55 0.005 0.225

0.30 0.12 0.4

The column-sorted array is

设置输出宽度及左对齐

0.15 0.005 0.225

0.3 0.12 0.375

0.55 0.875 0.4



(11)通讯录排序输出。创建一结构体类型描述朋友信息，包括姓名（拼音或英文）、电话号码（字符串）、年龄（整数）。键盘输入 n ($n < 10$) 个朋友的信息存储到结构体数组（通讯录）中，按他们的年龄从大到小的顺序排序，若两人年龄相同，则进一步按姓名升序排序，输出排序后的通讯录